

```

1  Library IEEE;
2  Use IEEE.std_logic_1164.all;
3  Use IEEE.numeric_std.all;
4
5  Entity ExecUnit is
6      Generic ( N : natural := 64 );
7      Port (
8          A      : in  std_logic_vector(N-1 downto 0);
9          B      : in  std_logic_vector(N-1 downto 0);
10         FuncClass : in  std_logic_vector(1 downto 0);
11         LogicFN  : in  std_logic_vector(1 downto 0);
12         ShiftFN  : in  std_logic_vector(1 downto 0);
13         AddnSub  : in  std_logic := '0';
14         ExtWord  : in  std_logic := '0';
15         Y        : out std_logic_vector(N-1 downto 0);
16         Zero     : out std_logic;
17         AltB     : out std_logic;
18         AltBu    : out std_logic
19     );
20 End Entity ExecUnit;
21
22
23
24 Architecture Structural of ExecUnit is
25
26
27     component ArithUnit is
28         Generic ( N : natural := 64 );
29         Port (
30             A      : in  std_logic_vector(N-1 downto 0);
31             B      : in  std_logic_vector(N-1 downto 0);
32             AddnSub : in  std_logic;
33             ExtWord : in  std_logic;
34             S      : out std_logic_vector(N-1 downto 0);
35             Cout    : out std_logic;
36             Ovfl    : out std_logic
37         );
38     end component ArithUnit;
39
40     component ShiftUnit is
41         Generic ( N : natural := 64 );
42         Port (
43             A      : in  std_logic_vector(N-1 downto 0);
44             B      : in  std_logic_vector(N-1 downto 0);
45             ShiftFN : in  std_logic_vector(1 downto 0);
46             ExtWord : in  std_logic;
47             Y      : out std_logic_vector(N-1 downto 0)
48         );
49     end component ShiftUnit;
50
51     component LogicUnit is
52         Generic ( N : natural := 64 );
53         Port (
54             A      : in  std_logic_vector(N-1 downto 0);
55             B      : in  std_logic_vector(N-1 downto 0);
56             LogicFN : in  std_logic_vector(1 downto 0);
57             Y      : out std_logic_vector(N-1 downto 0)
58         );
59     end component LogicUnit;
60
61
62     signal ArithResult : std_logic_vector(N-1 downto 0);
63     signal ShiftResult : std_logic_vector(N-1 downto 0);
64     signal LogicResult : std_logic_vector(N-1 downto 0);
65
66
67     signal ArithCout    : std_logic;
68     signal ArithOvfl    : std_logic;
69

```

```

70
71     signal Sub_Result      : std_logic_vector(N-1 downto 0);
72     signal Sub_Cout       : std_logic;
73     signal Sub_Ovfl       : std_logic;
74
75     signal SltResult      : std_logic_vector(N-1 downto 0);
76     signal SltuResult     : std_logic_vector(N-1 downto 0);
77
78
79     signal Zero_int       : std_logic;
80     signal AltB_int       : std_logic;
81     signal AltBu_int      : std_logic;
82
83 Begin
84
85
86     ArithInst : component ArithUnit
87         Generic map ( N => N )
88         Port map (
89             A          => A,
90             B          => B,
91
92             AddnSub    => AddnSub,
93             ExtWord    => ExtWord,
94             S          => ArithResult,
95             Cout       => open,
96             Ovfl       => open
97         );
98
99
100    ShiftInst : component ShiftUnit
101        Generic map ( N => N )
102        Port map (
103            A          => A,
104            B          => B,
105            ShiftFN    => ShiftFN,
106            ExtWord    => ExtWord,
107            Y          => ShiftResult
108        );
109
110
111    LogicInst : component LogicUnit
112        Generic map ( N => N )
113        Port map (
114            A          => A,
115            B          => B,
116            LogicFN    => LogicFN,
117            Y          => LogicResult
118        );
119
120
121    SubInst : component ArithUnit
122        Generic map ( N => N )
123        Port map (
124            A          => A,
125            B          => B,
126            AddnSub    => '1',
127            ExtWord    => '0',
128            S          => Sub_Result,
129            Cout       => Sub_Cout,
130            Ovfl       => Sub_Ovfl
131        );
132
133
134    process(Sub_Result)
135        variable allzero : std_logic;
136    begin
137        allzero := '0';
138        for i in 0 to N-1 loop

```

```

139         allzero := allzero or Sub_Result(i);
140     end loop;
141     Zero_int <= not allzero;
142 end process;
143
144 AltB_int <= Sub_Result(N-1) xor Sub_Ovfl;
145
146
147 AltBu_int <= not Sub_Cout;
148
149 Zero <= Zero_int;
150 AltB <= AltB_int;
151 AltBu <= AltBu_int;
152
153 SltResult <= (N-1 downto 1 => '0') & AltB_int;
154
155
156 SltuResult <= (N-1 downto 1 => '0') & AltBu_int;
157
158
159 process(FuncClass, ShiftFN, ArithResult, ShiftResult,
160         LogicResult, SltResult, SltuResult)
161 begin
162     case FuncClass is
163         when "01" => Y <= LogicResult;
164         when "10" => Y <= SltResult;
165         when "11" => Y <= SltuResult;
166         when "00" =>
167
168             if ShiftFN = "00" then
169                 Y <= ArithResult;
170             else
171                 Y <= ShiftResult;
172             end if;
173         when others => Y <= (others => 'X');
174     end case;
175 end process;
176
177 End Architecture Structural;

```